Quarto Lecture Notes

Thomas Hudson

1 September, 2026

Table of contents

Preface

Welcome to a simple template for creating lecture notes using Quarto. This template is designed to be easy to use and modify, allowing you to focus on the content of your notes rather than the formatting.

This is the landing page, and is an html render of `index.qmd`, serving as the main entry point for the notes, behaving like a title page. You can naviagate to other chapters by clicking on the links in the sidebar, or at the bottom of the page. The first chapter will provide a quick start guide to quarto, and the rest of the chapters will be a simple template for you to use to create your own lecture notes. You can add as many chapters as you like, and modify the content and formatting to suit your needs.

Note that these notes assume that you are already very familiar with create LaTeX notes, and work from there. There will doubtless be errors and typos present in this template. If you do spot anything, please feel free to let me know via email.

Dr Thomas Hudson, Sept 2026

1 Introduction to Quarto

Quarto is a clever multi-output document rendering tool. From a single set of markdown source files, it allows the generation of a wide variety of output formats. For those familiar with LaTeX, it allows us to generate html-rendered notes alongside pdf notes that we are familiar with with relatively low overheads for conversion, which is why I first started using it.

From an accessibility point of view, there are several key issues with PDFs:

- PDF files lack the underlying tags which allow screenreaders to parse them correctly, and sometimes order material incorrectly to how it appears visually;
- PDF colour schemes can't be adjusted easily to help those who benefit from alternative colour contrasts; and
- PDFs are difficult to resize because the arrangement of content is static.

Even if none of your students need to use a screen reader, the use of html brings plenty of additional benefits, as I hope I'll convince you. Given how many of our students are accessing notes on a tablet, phone or other screen, a massive positive of html content is that it can dynamically resize to allow everyone to parse information more easily. It also allows us to embed dynamic content which makes lecture notes more engaging! If you need more persuasion, here's a quote from a lecture evaluation this year:

> *A special mention also goes to the online notes Hudson has provided for this module, which resize themselves sensibly to any screen size and are also contain a useful search bar that shows you context of where words appear. These are very cleanly presented, organised, fast, and overall excellently made. This is probably the first time I've not felt friction in using digital lecture notes over printed ones.*

If you are already familiar with rendering pdfs from LaTeX code, you will likely find that Quarto is relatively easy to adapt to. Just like LaTeX, we need to install a renderer, and much of the syntax and approach is very similar to the way LaTeX works, but there are some key tweaks to be made which I'll highlight.

1.1 Aims

By the end of this section you should:

- know what Quarto is;
- have it installed;
- have rendered this template to produce a PDF and HTML version of the notes; and
- know how to upload and display the rendered html and pdf notes on a Moodle page.

1.2 Getting Started

Let's begin by installing Quarto. You can find full installation instructions on the Quarto website, so follow these first. The easiest way to use Quarto is probably via RStudio; this provides you with a direct render button like some LaTeX editors. You can also use Quarto from the command line, and so using this method, it's compatible with many IDEs or just a plain text editor. I will use VSCode for the demo, as it's what I use on a daily basis, and the remainder of the instructions here proceed on this basis.

Once Quarto is installed successfully, next download the template repo used for these notes. To do this:

1. Go to the github repo for these notes, github.com/thudso/QuartoLectureNotesTemplate.
2. Click "Code" and then "Download ZIP" (if you are already a git user and know how, you could also just clone the repo).
3. Unzip the files into an appropriate directory and open the folder in your IDE of choice.
4. Using the terminal, navigate to the unzipped directory and run `quarto render` in the terminal to build the notes.
5. After some messages in the terminal, you should now see that a `docs` folder has appeared in the main directory, which contains the rendered notes in both PDF and HTML format.

If you look inside `docs`, you will see a single pdf file alongside individual html pages for each chapter. `index.html` is the main page, and you can navigate to the other pages using the menu on the left-hand side of the page. If you open `index.html` in a browser, you should see that the notes are fully rendered. The rendering is dynamic, so exactly what you see will depend on the device you are using and the window width; try resizing the window.

If you are using a full-width screen, you should see chapters listed on the left, and the content of the chapter on the right. The left sidebar also includes a dynamic search box, a pdf download button, and switch to change from light to dark mode. If you are using a smaller screen, the left sidebar will be hidden, but you can still access it by clicking the menu button in the top left corner of the page.

1.3 Editing the notes

To update notes, you can edit the `.qmd` files in the main folder using a text editor or your preferred IDE. Notice that each `.qmd` file corresponds to a `.html` file in the rendered notes. The `_quarto.yml` file is the header which sets the overall configuration for the notes. Inspecting this, you can edit this to change the title of the notes, the author, and various other settings. In this project, I have add some explanatory comments. If you add new chapters, you will need to add them to the `_quarto.yml` file in the `chapters` section.

After making edits, you can then re-render the notes using `quarto render` in the terminal, and the updated notes will be available in the `docs` folder. More details about Quarto syntax follow in Chapter 2.

One nice feature of Quarto is the ability to use a dynamic preview of the html notes as you edit them. To activate this, run `quarto preview .` in the terminal, and a new browser window will open with the notes. As you edit and save changes to the `.qmd` files, the browser pge will automatically update to show your changes. This is a very useful feature for checking that your edits are correct before rendering the final version of the notes. Note that the `preview` command will not render a pdf, which is slower. This means you should always run `quarto render` to check the pdf version of the notes before uploading them to Moodle.

1.4 Uploading to Moodle

To make the notes available on Moodle, follow the following steps:

1. Render the notes using `quarto render` in the terminal, running in the folder containing `_quarto.yml`.
2. Zip the `docs` folder.
3. On your Moodle page, click "Add an activity or resource" and select "File".
4. Give the file a name (e.g., "Lecture Notes"), and upload the zipped `docs` folder by dragging and dropping it into the file upload box.
5. Once the zip file is uploaded, click on it in the box, then click the `unzip` button on the dialogue that appears to extract the contents of the zip file.
6. You should now see a folder called `docs` in your Moodle course: click on it. It will open, and click again on `index.html`. In the dialogue that appears, click `Set main file`. This will ensure that when students click on the link to the notes, they will be taken to the main page of the notes.
7. Now click "Save and return to course" at the bottom of the page.

After returning to the main Moodle page, you should now find that the link to the notes is available, and clicking on it will open the html notes in a new tab in the browser. You

will probably also want to provide a pdf version of the notes for students to download. To add a direct link to the pdf version of the notes for students to download, follow the following steps:

1. On your Moodle page, click "Add an activity or resource" and select "File".
2. Give the file a name (e.g. "PDF Lecture Notes").
3. Click on "Plus" button just above the "Select files" section, and then navigate to the `docs` folder you just uploaded, then to the `QuartoLectureNotesTemplate.pdf` file.
4. Next, change the radio buttons at the top from `Make a copy of the file` to `Link to the file`. This will ensure that when you update the notes, the pdf link will always point to the latest version of the notes, and you only need to update the notes in one place.
5. Finally, click `Select this file` to add the pdf link to your Moodle page.

After these initial steps, if you re-render the notes and want to update the version on Moodle, you now simply zip and upload the `docs` folder again, and the pdf link will automatically point to the latest version of the notes, as long as you did not do anything to change the name of `index.html` or the overall project.

1.5 What if I have existing notes?

Quarto is great if you are writing a fresh set of notes, as you can learn as you go. But what if you have existing notes for your module? The good news is that there are tools both old and new that will help you to make the conversion:

1.5.1 Option 1: Use AI to convert

Ferran and I have tested using AI to convert `.tex` files to `.qmd`. I tried prompting OpenAI's Sol 5.6 (available via NebulaONE), and it did an excellent job of converting a tex file and letting me know what it did. Ferran has used the Copilot agent in the past, and found it also did a good job of the conversion. This probably means that AI should be your first choice to convert existing, but other options follow below in case you are not keen on using AI agents.

1.5.2 Option 2: Use pandoc to convert

Pandoc is the format-to-format document conversion tool that is beneath a lot of what Quarto does, so installing pandoc and using it on the command line may get you some way. The following command:

```
pandoc old_notes.tex -s -o new_notes.md
```

will take `old_notes.tex` and output a markdown version `new_notes.md`. To try rendering in Quarto, simply change the extension from `.md` to `.qmd` and then render using `quarto render new_notes.md`. To make this a full project, you will need to add a `_quarto.yml` header file (or modify the existing header at the top of the file). Unfortunately, you will likely find that there is quite a lot of cleaning up of the underlying file to do after this, particular with a long, complex set of notes. I originally used this route to convert my existing MA4J1 Continuum Mechanics notes from LaTeX to Quarto Markdown in 2023, and it took me about a week to iron out all the issues, so there will definitely be work to be done!

1.5.3 Option 3: Manually convert

Given that even automatic conversion using pandoc is going to be fairly labour intensive, it may be simplest to manually convert your notes section by section, adapting the equation blocks to markdown syntax, changing section headings and figures, etc. Some clever use of find-and-replace with regular expressions can help to speed this up (again, an AI prompt to help design the regexes may help you to accelerate this process!).

2 An example chapter

2.1 Aims

By the end of this section you should be able to:

- Use sections and subsections to structure your notes;
- Understand basic Quarto syntax for writing text, equations, and code; and
- Include figures in your notes, and reference them in the text.

2.2 Sections and subsections

It's important to structure your notes using sections and subsections in an ordered fashion for accessibility. You can create a section by using a single `#` followed by the chapter title, and a section by using two `##` followed by the subsection title. You can also create subsections by using three `###` followed by the subsubsection title (and so on). Sections are labelled by including a label in curly braces after the section title, for example `# Example section {#sec-example-section}`. Looking at the markdown file will show you this clearly. It's important to note that the label should be unique, and should not contain spaces. For a section, labels should always begin with `sec-`. You can then reference the section using `@sec-ex-sections-and-subsections`, which renders to Section 2.2.

To create cross-references to sections, use the syntax `@sec-label`, where `label` is the label you have assigned to the section. The label for this section is `#sec-ex-sections-and-subsections`, so `@sec-ex-sections-and-subsections` is the equivalent of the latex syntax `\ref{sec-ex-sections-and-subsections}`, and renders to Section 2.2. Notice that by default, the link generated in the html will state the nature of the reference; this is partly why the `sec-` prefix is used.

2.3 Basic formatting

Text can be formatted in *italics* using a single asterisk, **bold** using double asterisks, or ***bold italics*** using triple asterisks. Fixed-width text (e.g. for code snippets) can be created using backticks, so `` `fixed width text` `` becomes `fixed width text`.

You can create lists using - for unordered lists, and numbers or letters with a full stop for ordered lists. You need a line of whitespace before a list to ensure it renders correctly, and tabs are used to create indents. A couple of example lists follow below:

Example List 1:

- Item 1
  - Item 1a
- Item 2

Example List 2:

1. Item 1
   a. Item 1a
2. Item 2

Links can be created using the syntax `[link text](url)`, for example Quarto website. For many further examples of formatting, check the Quarto documentation.

2.4 Equations

To render equations, you should use dollar sign and standard LaTeX syntax. For inline equations, you use a single dollar sign as usual, for example $\int_0^1 x^2 \, \mathrm{d}x = \frac{1}{3}$. For block equations, use double dollar signs, for example:

$$\frac{\partial u}{\partial t} = \Delta u. \tag{2.1}$$

Block equations are labelled using a label in curly braces *after* the equation, for example above, the markdown code is `$$...$$ {#eq-example-equation}`. To avoid creating an equation tag, simply omit a label.

You can reference an equation using `@eq-example-equation`, which renders to Equation 2.1. Note that the `eq-` prefix is used for equation labels, just as `sec-` preceded section labels.

It is also possible to use aligned and gathered equations, for example:

$$\begin{aligned} 4x + 2y &= 0, \\ 22x + 2y &= 3, \end{aligned} \tag{2.2}$$

or for gathered equations:

$$\begin{gathered} \text{Minimise the function} \quad f(x, y) := x^2 + 3xy + y^2 \\ \text{subject to the constraint } x + y = 1. \end{gathered} \tag{2.3}$$

Note that you can't label individual lines in a multi-line equation in Quarto, only the entire equation block. For accessibility purposes, this is a good thing, as it avoids confusion about which line is being referenced. You can always write a series of equations as separate blocks consecutively, and label them individually if you wish.

2.5 Figures

Including figures is relatively straightforward, but requires its own special syntax; an example figure is shown in Figure 2.1. Figures provide a first example of Quarto's triple colon `:::` syntax, which is used to surround custom blocks such as figures, code blocks, and include other special elements.

The image itself is included using the syntax `![](figs/figure.png){height=8cm fig-alt="..."}`, where:

- `figs/figure.png` is the path to the figure file;
- `height=8cm` sets the height of the figure (other formatting options are available, see the Quarto documentation);
- `fig-alt="..."` provides alternative text for the figure. Including a proper description of the figure is important for accessibility, as it allows screen readers to describe the figure to visually impaired users. The alternative text should be a concise description of the figure, and should not include any information that is already provided in the caption.

The caption for the figure is included in a block after the figure. Note that the figure opens with a triple colon and the label in curly braces `::: {#fig-geometry}`. Note once more that the label should begin `fig-`. Referencing the figure is done using the syntax `@fig-geometry`, which renders to Figure 2.1. The figure caption is included in the block after the figure, and should provide a concise description of the figure, as well as any relevant information about the source of the figure. Note that best practice is to keep the caption and alternative text distinct; the alt-text is for those who cannot see the image, providing supplementary information, whereas the caption is for those who can see the image, providing context and additional information.

2.6 Callout blocks

A tool that Quarto provides that may be useful in lecture notes are callout blocks. These come in 5 default types, see the Quarto documentation for details. All are created using a three colon block, and you can see the syntax in the `.qmd` source file for this chapter. Below follow some examples of their use.

Figure 2.1: A woman teaching geometry to a group of students; she is most likely the personification of the subject. Attributed to the Meliacin Master, an anonymous illuminator based in Paris; this version taken from Wikimedia Commons.

> **Note**
>
> Note that there are five types of callouts, including: `note`, `warning`, `important`, `tip`, and `caution`.

> **Warning**
>
> Look out! This is a warning.

> **Important**
>
> This is an 'important' block. Note that I can't really recommend callouts on accessibility grounds, as they don't automatically restyle themselves with good contrast when switching to dark mode; try switching for yourself and see what happens!

> **Tip with Title**
>
> This is an example of a tip callout block with a title.

> **Expand To Learn About Collapse**
>
> This is an example of a 'folded' caution callout that can be expanded by the user. You can use `collapse="true"` to collapse it by default or `collapse="false"` to make a collapsible callout that is expanded by default.

See the markdown code for examples of how to use these. Note that with basic Quarto, these can't be numbered or cross-referenced, but some extensions have been developed to allow this, see for example Ute Hahn's github repository here.

2.7 Citations and references

It is possible to use BibTeX to manage and render references correctly and automatically just as you would in LaTeX. Citations can be produced using `@bibtexkey`. Using the basic bib file provided and the AMS numeric style option set in the the YAML header, `@rudin1976principles` renders as [1].

You can include your own bibliography by setting the `bibliography` option in the `_quarto.yml` file. Note that to make the bibliography render properly, you need to include a special block

```
::: {#refs}
:::
```

somewhere in your document where you want the bibliography to appear. Here, the rendered version appears below for convenience.

I have not used referencing myself, so I recommend see the appropriate page in the Quarto docs for further details.

References

[1] Rudin, Walter, *Principles of mathematical analysis*, 3rd ed., McGraw-Hill, New York, 1976.

2.8 Including code and video

These notes do also contain an advanced chapter which involve code elements, which is not rendered by default. The reason for this is that the software overheads to get this working are a little higher, and I don't expect everyone to need these features. If you'd like to explore using code in your notes, follow the steps below:

1. In `_quarto.yml`, uncomment line 14 of the file, i.e. change `# - chapter3.qmd` to `- chapter3.qmd`.
2. Further information about how to configure Quarto correctly to work with Python in the Quarto docs. I can't provide full instructions for doing this, as it will depend on what level of access you have on your machine; hopefully if you are the sort of person who is happy to dabble with these things, you will find it fairly straightforward. Note that integration with R, Julia and JavaScript are also all possible. To render the next chapter, you will need Quarto to be able to access a version of python which has `numpy`, `matplotlib`, `linalg` and `qrcode` installed.
3. Once this is done, rerender the document. You should find the error messages descriptive if there are issues.